



ELEC3609/9609 - Week 6 Lab

05 Django Authentication and File Storage System

In this lab, you'll continue developing your TIL web application with a focus on user authentication, session handling, file storage, and system-level logic such as signals. You'll also explore key concepts in secure state management, model customisation, and global context sharing in templates. While not every topic will be deeply examined, this lab aims to broaden your understanding of how modern web frameworks manage users, sessions, and uploaded data.

Individual Task: Individual Task 5 - Understand Authentication, Sessions, and Cookies

Lab Resource: TIL-Lab05 (Today I Learned project) source code.

Learning Objectives

After completing this lab, students should be able to:

- Explain how HTTP, cookies, and sessions interact in a Django application.
- Understand and use Django's built-in authentication system with custom user models.
- Implement secure file upload and storage handling using Django's media configuration.
- Recognise the use and trade-offs of signals and context processors in real-world applications.
- Navigate Django's session framework and apply context-aware template rendering.

Chapter 1. HTTP Recap

HTTP (HyperText Transfer Protocol) governs communication between a client and a server. It is **stateless**, meaning each request is independent, and the server does not retain memory of previous interactions.

Client-Server Model

The client (e.g., a browser or app) initiates a request, while the server processes and responds.

Stateless Nature

Each HTTP interaction is isolated. The server does not retain knowledge of earlier requests unless external mechanisms (e.g., cookies, sessions) are used.

OSI Model

The OSI model describes seven layers of network communication. HTTP runs at the **application layer**, the topmost level, while data transmission is handled by the **transport layer** (usually TCP). These layers are independent, HTTP focuses on content logic, while TCP ensures reliable delivery.

Application Layer

Web protocols like HTTP, FTP, and DNS operate at the application layer. This layer handles **what** is being communicated (e.g., web pages, data formats), not **how** it is sent. Below it, the transport layer (e.g., TCP) handles actual data delivery.

HTTP Methods & Server Status Codes

HTTP Method	Description	Status Code	Meaning
GET	Retrieve data	200	OK (Success)
POST	Submit new data	404	Not Found
PUT	Replace existing data	500	Internal Server Error
DELETE	Remove data	403	Forbidden
PATCH	Partially update data	301	Moved Permanently

Headers

Headers are metadata sent with requests and responses. Common examples include:

- `Content-Type: application/json` — defines the format of the request or response

body.

- `Set-Cookie` — manages state across stateless HTTP.
- `Authorization` — carries access credentials or tokens.

Chapter 2. Cookie & Session

All communication between the client and server is done through the HTTP protocol, a stateless protocol. This means that one state is not related to another (independent). This requires the computer on client side to establish a TCP connection to the server every time it makes a request.

Without a persistent connection between the client and server, the software on each side (endpoint) cannot rely solely on the TCP connection to hold a state or hold session state. TCP connection itself is usually short-lived.

2.1 What does holding a state mean?

Suppose that you want to access a page A on a website that requires users to be logged in. Then, you log in to the website and successfully access page A. When you want to move to page B on the same website, without a process of holding a state, **you will be prompted to log in again**. This will happen every time you access a different page on the same website.

The process of informing "who" is currently logged in and storing that information is the basis of a session (a semi-permanent exchange of information). It is difficult to make HTTP hold a state (because HTTP is a stateless protocol). Therefore, other techniques are needed to address this issue, namely **cookies** and **sessions**.

2.2 How to Hold a State?

Since HTTP is stateless, web servers need a way to remember users across multiple requests. The most common solution is using **sessions** in combination with **cookies**.

How It Works:

1. When a user logs in, the server creates a **session ID**, a random token identifying that user.

2. This session ID is stored:
 - As a **cookie** in the user's browser.
 - On the **server**, linked to the user's actual data (e.g., username).
3. On each request, the browser automatically sends the cookie (with the session ID).
4. The server reads the session ID and retrieves the corresponding data from its storage (memory or database).
5. This way, the server "remembers" who you are, without storing any personal data in the browser.

**TIP**

A cookie is stored on the client side, while the session data stays on the server. Only the session ID is stored in the cookie, not the actual user data. This keeps the system secure and efficient.

Why This Is Safer:

- Cookies have a **4KB size limit**.
- Storing sensitive data in cookies is risky (can be stolen).
- Sessions keep user data on the server, **more secure** and **scalable**.

2.3 Django Session

Django provides full support for anonymous sessions. Django sessions allow you to store and retrieve arbitrary data on a per-site-visitor basis. It works by saving session data on the server side and associating it with a session ID stored in a cookie on the client-side. Unlike cookies, session data is stored securely on the server and can hold more complex data than simple key-value pairs.

- **Session Creation:** When an authenticated user accesses the site, Django assigns them a unique session ID stored in a cookie (`sessionid`). This cookie is sent with every request.
- **Data Storage:** Django stores session data server-side, and you can access this session data via `request.session`.
- **Session Expiry:** Django automatically manages session expiry based on your configuration.

Enable Django Sessions

Sessions are implemented via a piece of **middleware**. Sessions are enabled in Django if `django.contrib.sessions.middleware.SessionMiddleware` is added to your `MIDDLEWARE` setting.

! INFO

The default `settings.py` created by `django-admin startproject` has `SessionMiddleware` activated.

Configure Django Sessions

By default, Django stores sessions in your database (using the model `django.contrib.sessions.models.Session`). Though this is convenient, in some setups it's faster to store session data elsewhere, so Django can be configured to store session data on your filesystem or in your cache:

```
# settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.db' # database-backed
sessions (default)
SESSION_ENGINE = 'django.contrib.sessions.backends.cache' # cached sessions
SESSION_ENGINE = 'django.contrib.sessions.backends.file' # file-based
sessions
```

💡 TIP

Read more about [Configuring the session engine](#).

There are also a few **Django settings** giving you control over session behaviour:

```
# settings.py
# https://docs.djangoproject.com/en/stable/ref/settings/#sessions
SESSION_COOKIE_AGE = 1800 # Set session lifetime to 1800
seconds
SESSION_EXPIRE_AT_BROWSER_CLOSE = True # Expire the session when the
browser closes
```

Use Django Sessions

With Django's session framework, it is easy to manage user session data and display critical

information. The following function demonstrates how to use Django's session framework to track and display remaining session time and last login time for authenticated users:

```
# til_app/context_processors.py
from django.utils import timezone

def remaining_session_time(request):
    # Check if user is authenticated
    if request.user.is_authenticated and request.session:
        # Get remaining session time
        remaining_time = request.session.get_expiry_age()

        # Store and access last login time
        last_login = request.session.get('last_login', None)
        if not last_login:
            last_login = timezone.localtime(timezone.now()).strftime('%m/%d/%Y - %I:%M%p')
            print(last_login)
            request.session['last_login'] = last_login

        return {'remaining_time': remaining_time, 'last_login': last_login}
    else:
        return {'remaining_time': None, 'last_login': None}
```



GLOBAL DATA WITH CONTEXT PROCESSORS

In TIL app, we use a context processor to hold function that returns a dictionary. This dictionary is then merged into the context of every template rendering. Simply but, context processors allow you to add common data available globally to all templates without repeating code in every view. Read more about [Context processors](#).

Chapter 3. Django Authentication System

Django provides a robust and customisable user authentication system that is built into its core framework. This configuration has evolved to serve the most common project needs, handling a reasonably wide range of tasks, and has a careful implementation of passwords and permissions.



INFO

Django's authentication system handles both **authentication** (verifying user identity) and **authorisation** (determining user permissions).

In this TIL project, we will create customised user model to use email for authentication and registration method. Read more about [How to use sessions](#), [Django built-in authentication system](#) and [Customising authentication in Django](#).

3.1 Substitute Customised User Model

Substituting the default user model in Django is necessary when you need to use email instead of username for login, store additional fields like profile pictures or roles, or customise user behavior. It provides flexibility for handling specific authentication and user management needs without having to create additional models or tables.

Step 1: Update settings

In Django project's `settings.py`, we need to define the custom user model by setting the `AUTH_USER_MODEL` variable to point to our custom model.

```
# settings.py
AUTH_USER_MODEL = 'til_app.CustomUser'
```

This tells Django to use custom user model instead of the default `User` model.

SUBSTITUTE `User` CAN BE COMPLEX

Changing `AUTH_USER_MODEL` after you've created database tables is possible, but can be complex. This changes cannot be done automatically and requires manually fixing your schema, moving your data from the old user table, and possibly manually reapplying some migrations. See [#25313](#) for an outline of the steps.

Step 2: Creating a Custom User Manager

A custom user manager is needed to properly create user instances. This manager handles the creation of both regular users and superusers. We override the `create_user` and `create_superuser` methods to ensure that users are created correctly.

til_app/models.py

```
class CustomAccountManager(BaseUserManager):
    def create_user(self, email, password=None):
        if not email:
            raise ValueError(_('You must provide an email address'))

        email = self.normalize_email(email)
        user = self.model(email=email)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None):
        user = self.create_user(email, password)
        user.is_admin = True
```

models.py

```
class CustomUser(AbstractUser, PermissionsMixin):
    username = models.CharField(max_length=30, unique=False) # Username is
not unique
    email = models.EmailField(
        _("email address"),
        max_length=255,
        unique=True,
        error_messages={
            "unique": _("A user with that email address already exists."),
        },
    )
    image_location = models.CharField(
        max_length=255,
        default="/til_app/media/default_user_logo.jpg",
        unique=False,
        null=True,
        blank=True
    )
    is_admin = models.BooleanField(
        _("admin status"),
        default=False,
        help_text=_("Designates whether the user can log into this admin
site.")
    )
    is_active = models.BooleanField(
        _("active"),
        default=True, # Default is False, can be changed to True based on
```

your needs

```

        help_text=_("Designates whether this user should be treated as
active. Unselect this instead of deleting "
                    "accounts.")
    )
    date_joined = models.DateTimeField(_("date joined"),
default=timezone.now)

    objects = CustomAccountManager()

EMAIL_FIELD = "email"
USERNAME_FIELD = "email" # Set the email as the unique identifier
REQUIRED_FIELDS = [] # Email and password are required by default

def clean(self):
    super().clean()
    self.email = self.__class__.objects.normalize_email(self.email)

def __str__(self):
    return self.email # Display the email instead of the username

@property
def is_staff(self):
    return self.is_admin

```

► Code Explanation

Step 4: Update Admin Interface

To manage the custom user model in the Django admin interface, create a custom admin class.

```

# admin.py

class UserAdmin(BaseUserAdmin):
    # The forms to add and change user instances
    form = UserChangeForm
    add_form = UserCreationForm

    # The fields to be used in displaying the User model.
    # These override the definitions on the base UserAdmin
    # that reference specific fields on auth.User.
    search_fields = ('email', 'username')

```

```
list_filter = ('email', 'username', 'is_active', 'is_admin')
list_display = ('email', 'username', 'is_active', 'is_admin')

fieldsets = (
    (None, {'fields': ('email', 'username', 'image_location',)}),
    ('Permissions', {'fields': ('is_admin', 'is_active')}),
)
add_fieldsets = (
    (None, {
        'classes': ('wide',),
        'fields': (
            'email', 'username', 'image_location', 'password1',
            'password2', 'is_active', 'is_admin')
    }),
)

ordering = ('date_joined',)

admin.site.register(CustomUser, UserAdmin)
```

► Code Explanation



TIP

This UserAdmin makes the Django admin UI compatible with your `CustomUser` model. Without it, you'd still have your `CustomUser` in the database, but managing it through the admin site would be very clunky (or even impossible). View more in `admin.py`.

Step 5: Migrate the Database

Everytime updating the model schema, run the migrations to apply the changes to the database.

```
python manage.py makemigrations
python manage.py migrate
```

⚠ WARNING

If you already have migrations with the default `User` model, switching to a custom user

model can cause migration conflicts. It is recommended to start with a custom user model from the beginning. Or you might need to remove the database file `rm db.sqlite3` and apply migrations.

💡 **EXTEND `User` MODEL**

You might want to extend the existing `User` model without substituting your own model. You can create a **proxy model** based on `User`. If you wish to store information related to `User`, you can use a `OneToOneField` to a model containing the fields for additional information. Read more about [Extending the existing User model](#).

⚠️ **CUSTOM AUTHENTICATION BACKEND**

However, with `OneToOneField` to an additional model, if you want to change the authentication method to use email by default (instead of username), you need to write a custom authentication backend. Read more about [Writing an authentication backend](#).

3.2 Create Registration Form and Function

Django provides a powerful form handling framework that simplifies the creation, validation, and processing of forms in web applications. The `forms` module in Django allows developers to easily manage user input and perform validation without writing repetitive boilerplate code.

Key Advantages of Django Forms:

- **Built-in Validation:** Django forms provide built-in validation for different types of form fields. This ensures that the data submitted by the user meets the expected format and constraints.
- **Automatic HTML Generation:** Django forms can automatically generate HTML for form fields, including appropriate input types and attributes, reducing the need for manual HTML coding.
- **Security Features:** Django forms help protect against common web vulnerabilities like Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) by default.
- **Customisable and Extendable:** Django forms can be customised extensively. You can add your own validation logic, widgets, and error messages.

**TIP**

Read more about [Working with forms](#).

In this lab, we will create a custom user registration form using Django forms, including custom validation logic, and set up a registration view to handle user registration.

Step 1: Define RegisterForm

This is a specialised form for user registration. It extends

`django.contrib.auth.forms.UserCreationForm` with basic user creation functionality, and includes additional custom validation for email format and other rules.

```
# til_app/forms.py

class RegisterForm(UserCreationForm):
    email = forms.EmailField(
        max_length=255, help_text="Required add a valid email address.")

    class Meta:
        model = CustomUser
        fields = ("email", "password1", "password2")

    def save(self, commit=True):
        user = super(RegisterForm, self).save(commit=False)
        user.email = self.cleaned_data['email']
        user.username = "user#" + user.email
        if commit:
            user.save()
        return user

    def clean_email(self):
        email = self.cleaned_data["email"].lower()

        # simple uni email validation
        try:
            pattern = r"^[a-z]{4}[0-9]{4}@uni\.sydney\.edu\.au$"
            if not re.fullmatch(pattern, email):
                raise forms.ValidationError(
                    _("Please provide a valid uni email"))
        except Exception:
            raise forms.ValidationError(
                _("Please provide a valid uni email"))
```

```

# email uniqueness verification
try:
    CustomUser.objects.get(email=email)
except CustomUser.DoesNotExist: # Model-Level exception
    return email
raise forms.ValidationError(
    _("A user with that email already exists. "))

def clean_password2(self):
    password1 = self.cleaned_data.get("password1", "")
    password2 = self.cleaned_data["password2"]
    if password1 != password2:
        raise forms.ValidationError(
            _("The two password fields didn't match. "))
    return password2

```

► Code Explanation

Step 2: Define `register()` View

The registration view will handle both GET requests to display the form and POST requests to process the form submission.

```

# views.py

def register(request):
    form = RegisterForm()

    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid(): # Check if the form is valid
            user = form.save() # Save the user to the database

            # Authenticate the user
            email = form.cleaned_data.get('email')
            password = form.cleaned_data.get('password1')
            user = authenticate(request, email=email, password=password)

            if user is not None: # Check if authentication was successful
                login(request, user) # Log the user in
                messages.success(request, 'Your account has been
                successfully created and you are now logged in!')
                return redirect('index')

```

```
return render(request, 'registration/register.html', {'form': form})
```

! INFO

This example also shows how to log a user in. You typically use both `authenticate()` and `login()`. When a user logs in, the user's ID and the backend that was used for authentication are saved in the user's session. This allows the same **authentication backend** to fetch the user's details on a future request.

`authenticate()` checks the user's credentials against the authentication backends defined in Django's settings and returns a `User` object if valid. If the credentials are correct, `login()` saves the user's ID in the session, using Django's session framework. Read more about [How to log a user in](#).

Step 3: Create the Registration Template

The registration template(`til_app/templates/registration/register.html`) contains the registration form and dynamically handles any form errors.

```
<!-- templates/registration/register.html -->

<div class="login-page">
  <div class="form">
    <!-- Registration Form -->
    <form class="register-form" action="{% url 'register' %}" method="post">
      {% csrf_token %}
      <input type="email" name="email" id="email" placeholder="Email"
required autofocus>
      <input type="password" name="password1" class="input_section"
id="password" placeholder="Password" required>
      <input type="password" name="password2" class="input_section"
id="password" placeholder="Confirm Password" required>
      <button type="submit">Create</button>
      {% for field in form %}
        <p>
          {% for error in field.errors %}
            <p style="color:red">{{ error }}</p>
          {% endfor %}
        </p>
      {% endfor %}
    {% if form.non_field_errors %}
```

```

        <div style="color:red">
            <p>{{form.non_field_errors}}</p>
        </div>
    {% endif %}
    <p class="message">Already registered? <a href="{% url 'login'
%}">Sign In</a></p>
</form>
</div>
</div>

```

**TIP**

Instead of manually rendering each input field, you can try **Django's form output styles**, like `{{ form.as_div }}` to render the entire form with error handling automatically.

**INFO**

If you render a bound Form object (initialised with data), the act of rendering will automatically run the form's validation if it hasn't already happened, and the HTML output will include the validation errors as a `<ul class="errorlist">` near the field.

Step 4: Update URLs

Ensure that application `urls.py` is correctly configured to include the `register` view.

```

# urls.py

urlpatterns = [
    path('register/', views.register, name='register'),
    # ...,
]

```

3.3 Implement Login/Logout Function

Django provides built-in views for handling user authentication: `LoginView` for logging in and `LogoutView` for logging out. These views are robust, secure, and easy to integrate.

**DJANGO BUILT-IN AUTHENTICATION VIEWS**

In previous step, we introduced `authenticate()` to login automatically after successful registration. However, unless you are writing your own authentication system, you probably won't use this. Read more about [LoginView](#) and [LogoutView](#).

Step 1: Import Built-in Login and Logout Views.

```
# urls

from django.contrib.auth.views import LoginView, LogoutView

urlpatterns = [
    path('login/', LoginView.as_view(), name='login'),
    path('logout/', LogoutView.as_view(), name='logout'),
    # ...
]
```

To use built-in `auth.views.LoginView`, you need to provide a template named `registration/login.html` by default, unless you specify a different template name in the `LoginView` configuration:

```
urlpatterns = [
    path('login/', LoginView.as_view(template_name='my_custom_login.html'),
        name='login'),
    # ...
]
```

Step 2: Setup Templates

In the `login.html` template, even with `email` set as the unique identifier (`USERNAME_FIELD`), the input field must use `name="username"` because Django's authentication system expects it. This ensures the field correctly maps to `email` for login while remaining compatible with Django's default `LoginView`.

```
<!-- templates/registration/login.html -->

<div class="login-page">
  <div class="form">
    <!-- Login Form -->
    <form class="login-form" action="{% url 'login' %}" method="post">
```

```

{% csrf_token %}
<input type="text" name="username" placeholder="Email" required />
<input type="password" name="password" placeholder="Password" required
/>

<button type="submit">Login</button>
{% for field in form %}
    <p>
        {% for error in field.errors %}
            <p style="color:red">{{ error }}</p>
        {% endfor %}
    </p>
{% endfor %}
{% if form.non_field_errors %}
    <div style="color:red">
        <p>{{ form.non_field_errors }}</p>
    </div>
{% endif %}
<p class="message">Not registered? <a href="{% url 'register'
%}">Create an account</a></p>
</form>
</div>
</div>

```

And the logout form in base.html uses Django's built-in `LogoutView`, requiring `method="post"` and `{% csrf_token %}` for security against CSRF attacks.

```

<!-- templates/base.html -->

<form method="post" action="{% url 'logout' %}">
    {% csrf_token %}
    <button class="btn btn-danger" type="submit">Logout</button>
</form>

```

Step 3: Configure Login and Logout Redirects

In `settings.py`, `LOGIN_URL` specifies where users are redirected if they need to log in (e.g., `"/til_app/login"`). `LOGIN_REDIRECT_URL` defines where users go after a successful login (e.g., `"/til_app"`), and `LOGOUT_REDIRECT_URL` sets the landing page after logging out (e.g., `"/til_app"`). These settings manage user navigation and flow during authentication.

```
# settings.py
```

```
LOGIN_URL = "/til_app/login"
LOGIN_REDIRECT_URL = "/til_app"
LOGOUT_REDIRECT_URL = "/til_app"
```

3.4 Restrict Access to The Main Page

Django provides several methods to restrict access to views, ensuring only authorised users can access certain parts of the application. The most common way to restrict access is by using the `@login_required` decorator. This lab and TIL app will focus on `@login_required`:

```
from django.contrib.auth.decorators import login_required

@login_required
def my_view(request): ...
```

`login_required()` does the following:

- If the user isn't logged in, redirect to `settings.LOGIN_URL`, passing the current absolute path in the query string. Example: `/til_app/login/?next=/til_app/posts/`.
- If the user is logged in, execute the view normally. The view code is free to assume the user is logged in.

! INFO

Read more about [Limiting access to logged-in users](#).

3.5 Connect Post Model to CustomUser Model

Since `AUTH_USER_MODEL` setting has been changed to the custom user model, referencing `User` directly will not work in projects. You can directly import your custom user model `CustomUser`, however, it is generally best to use `get_user_model()` for flexibility, maintainability, and adherence to Django's best practices.

```
# models.py

class Post(models.Model):
```

```
subject = models.CharField(max_length=65, null=False)
contents = models.TextField(max_length=1000, default="", null=False)
creator = models.ForeignKey(CustomUser, on_delete=models.CASCADE)
created_at = models.DateTimeField(auto_now_add=True)
visibility = models.BooleanField(default=True)
tags = models.ManyToManyField(Tag, related_name='posts', blank=True) #
Many-to-many relationship
image = models.ImageField(upload_to='post_images/', blank=False,
null=False) # ImageField to handle images
```

The final TIL app for this lab provides a basic web framework with the following features:

- **User Authentication:** Users can register with an email, log in, and log out.
- **Post Management:** The main posts view is integrated with a database, allowing users to create and remove posts via their profile.
- **Comments and Tags:** Users can comment on posts and associate posts with tags.
- **Session Management:** A session expiration countdown is implemented, with automatic logout upon session expiry.

This setup ensures core functionalities for user interaction, content management, and session control.

! INFO

This lab focuses on the overall structure and integration. If you're interested in how each feature is implemented behind the scenes, feel free to explore the code, we won't go into every detail here.

💡 TIP

Read more about [Making queries](#) to learn how to create/retrieve/update/delete objects with a database-abstraction API (QuerySets).

Chapter 4. Django File Storage System

Django provides a robust file storage system that helps handle file uploads and manage media files (like images and documents). When a user uploads a file, Django allows you to save it to a local directory or even external storage (like AWS S3 or Google Cloud).

4.1 Django File Storage API

This TIL app uses Django File Storage API, configuring local file storage. Read more about [File storage API](#) and [Managing files](#).

Step1: Setup Basic File and Image Uploads

Django has built-in support for handling file uploads through `FileField` and `ImageField`. The following example demonstrates how to use an `ImageField` to store images uploaded by users.

```
# models.py

class Post(models.Model):
    # ...
    image = models.ImageField(upload_to='post_images/', blank=False,
                             null=False) # ImageField to handle image uploads
```

! INFO

Django's `ImageField` requires the [Pillow](#) library, uses `FileExtensionValidator` to validate that the file extension is supported by Pillow. Pillow is a popular Python Imaging Library (PIL) fork that supports opening, manipulating, and saving various image file formats.

The `ImageField` will automatically handle the uploaded image file and store it in the specified folder (`post_images/` in this case) within the media directory. The file will be uploaded using a form and saved with the model instance.

```
# forms.py

class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ['subject', 'contents', 'tags', 'image']
```

Step 2: Handle File Uploads in Views

Then you can handle file uploads in a Django view:

```
def create_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST, request.FILES) # request.FILES to
        handle uploaded files
        if form.is_valid():
            form.save()
            return redirect('post_list')
    else:
        form = PostForm()
    return render(request, 'create_post.html', {'form': form})
```

Remember to update application `urls.py` settings, and create a template.

```
<!-- create_post.html -->
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload Post</button>
</form>
```

! INFO

The above two code snippets serve as a guideline. In TIL app, post creation is associated with tags and implemented using AJAX. For more details, refer to

`til_app.views.create_post()` and `til_app/templates/profile.html`.

4.2 Default Behaviour with `FileSystemStorage` Class

Django's default `FileSystemStorage` does not **overwrite** files with the same name. Instead, it appends a suffix (e.g., `_1`, `_2`, etc.) to the filename to avoid overwriting an existing file. This ensures that previously uploaded files are not lost when a file with the same name is uploaded again.

If you need to overwrite the existing file with the same name, there are two main approaches:

Method 1: Manually Delete and Save the File:

Call `FileSystemStorage.delete()` before saving the new file.

```
# views.py

from django.core.files.storage import FileSystemStorage

storage = FileSystemStorage()

def save_avatar(instance, file):
    file_name = "profile.jpg"
    if storage.exists(file_name):
        storage.delete(file_name) # Delete existing file
    storage.save(file_name, file) # Save new file
```

Method 2: Create a Custom Storage Class:

You can extend `FileSystemStorage` and override the `get_available_name()` method to prevent appending a suffix, ensuring that files with the same name are overwritten.

```
# storage.py

from django.core.files.storage import FileSystemStorage

class OverwriteStorage(FileSystemStorage):
    def get_available_name(self, name, max_length=None):
        # If the filename already exists, remove it as if it was a true file
        system
        if self.exists(name):
            self.delete(name)
        return name
```

And you need to use this custom storage class in your models (as a **callable** to modify the used storage at runtime) when defining file or image fields.



TIP

Read more about [How to write a custom storage class](#).

4.3 Manage File Storage

```
# models.py
from .storage import OverwriteStorage

class Thing(models.Model):
    image = models.ImageField(
        max_length=SOME_CONST, storage=OverwriteStorage(),

# settings.py

# Django Media Files configuration
# https://docs.djangoproject.com/en/stable/ref/settings/#media-root
MEDIA_URL = '/til_app/media/'

MEDIA_ROOT = os.path.join(BASE_DIR, 'til_app/media')
```

In **development**, Django serves files directly from the local filesystem. To enable this, add the following to project `urls.py`:

```
# til_project/urls.py

urlpatterns = [
    # URL patterns
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

This configuration tells Django to serve files from the `MEDIA_ROOT` directory when accessed via `MEDIA_URL`.

THIS IS NOT SUITABLE FOR PRODUCTION USE!

The django's development server is not optimised for serving static and media files. It can lead to security problems and performance/concurrency bottleneck.

In production environments, media and static files should be served through a dedicated web server like Nginx or Apache or via a Content Delivery Network (CDN), which is designed for high-performance file delivery and efficient caching mechanisms.

For some common deployment strategies, see **[How to deploy static files](#)**.

Chapter 5. Signals and Override

Django includes a “signal dispatcher” which helps decoupled applications get notified when actions occur elsewhere in the framework. In a nutshell, signals allow certain senders to notify a set of receivers that some action has taken place. They’re especially useful when many pieces of code may be interested in the same events.

For example, register the signals by importing `signals.py` in the ready method.

```
# apps.py

class TilAppConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'til_app'

    def ready(self):
        import til_app.signals
```

❗ DJANGO SIGNALS ARE A POWERFUL FEATURE, BUT ALSO A CONTROVERSIAL ONE.

Some developers argue that signals increase complexity and make the codebase harder to trace and maintain. Alternatives like [custom model managers](#) or [overriding model methods](#) are often recommended for more explicit logic.

That said, understanding how signals work is still valuable. Even if you don’t use them in practice, they can give you insight into how Django’s internal event system and extensibility are designed.

The following reading is about two short examples of signal usage. Read more about [Django Signals](#).

5.1 Remove Image from File Storage When Deleting a Post

The file stored in the file system and the database record are considered separate entities (`ImageField` to `varchar`). When a model instance is deleted, Django only deletes the record from the database but not the associated file from the file system.

To delete the associated image file from storage when a post is deleted, you can override the `delete` method in the `Post` model or use a signal to handle this cleanup.

Option 1: Use Django Signals for Post Deletion

Use the `post_delete` signal to handle file deletion whenever a post is deleted.

```
# til_app/signals.py

from django.db.models.signals import post_delete
from django.dispatch import receiver

@receiver(post_delete, sender=Post)
def delete_post_image(sender, instance, **kwargs):
    if instance.image:
        instance.image.delete(False) # Deletes the file without updating or
        saving the model
```

Option 2: Override the `delete` Method in the `Post` Model

Override the `delete` method in the model to ensure that the associated image file is removed when the post is deleted.

```
class Post(models.Model):
    # ... other fields

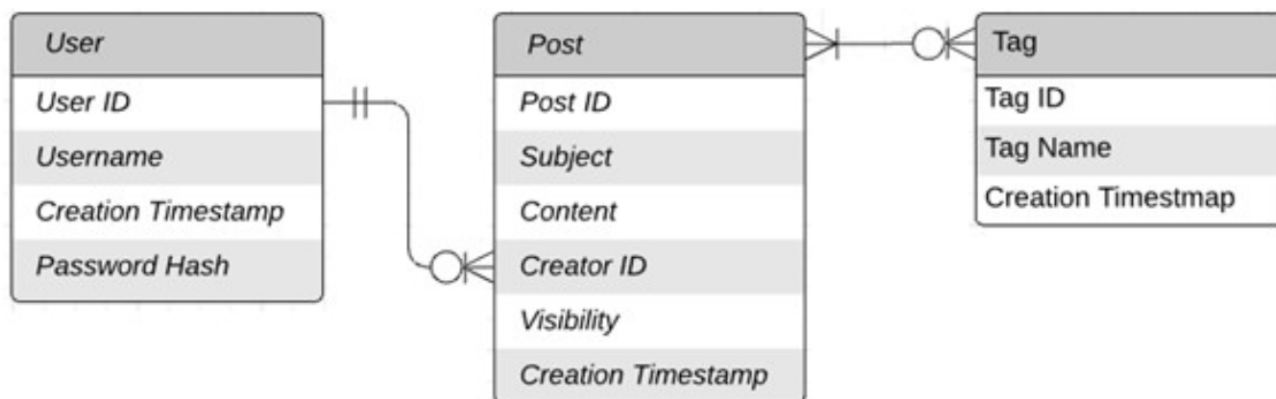
    def delete(self, *args, **kwargs):
        # Deletes the file associated with this instance using the storage
        backend.
        self.image.delete(save=False) # Without saving the instance again.

        # Calls the parent delete method to delete the instance from the
        database.
        super().delete(*args, **kwargs)
```

5.2 Handle Tag Creation in Django Admin

Recall the TIL diagram below, which models the relationship between `Post` and `Tag`, where a `Post` can be associated with **zero, one or many** `Tag`s. A `Tag` can be applied to **one or many**

Posts.



By default, `ManyToManyField()` does not support **at least one** constraint, which means Django does not restrict you from creating a `Tag` without any associated posts.

Override the `clean` Method in the `Tag` Model

The `clean()` method is a validation hook. It checks whether a `Tag` has at least one associated `Post`.

```
# models.py

from django.core.exceptions import ValidationError

class Tag(models.Model):
    name = models.CharField(max_length=10, unique=True)
    creation_date = models.DateTimeField(auto_now_add=True)

    def clean(self):
        if not self.posts.exists(): # Check if the tag has no associated
            posts
            raise ValidationError("A tag must be associated with at least
            one post.")
```

⚠ WARNING

In the built-in Django admin or form workflow, `clean()` is called before the object is saved. At that stage, the `Tag` has no primary key, so accessing `self.posts` raises a `ValueError` (not a `ValidationError`).

If you need to enforce this rule at creation time through a form, you must instead use a

`ModelForm.clean()` method, where the `posts` field is available from `cleaned_data`.

Create a `Tag` In Your Workflow

When creating tags inside your own workflow (e.g., during post creation), use `get_or_create()` to either fetch or create the `Tag` and attach it to the `Post`.

```
# views.py

new_tags = request.POST.get('new_tags', '').split(',')
for tag in new_tags:
    if tag.strip(): # Avoid empty tag inputs
        tag_obj, created = Tag.objects.get_or_create(name=tag.strip())
        post.tags.add(tag_obj)
```

❗ ENFORCE `clean()` IN MANUAL CREATION

`get_or_create()` calls `create()`, which does not trigger `clean()` automatically. If you want to enforce `Tag.clean()` in your own workflow, you should call it after the object has been created (so it already has a primary key). For example:

```
tag_obj.full_clean() # or tag_obj.clean()
```

This way, you avoid the `ValueError` (caused by no pk) while still enforcing your custom rule when appropriate.

Define a Signal Handler for `pre_delete`

Create a function that will be triggered whenever a `Post` instance is deleted. This function will check each tag associated with the deleted post to see if only current post is linked. If yes, it will delete the tag.

```
# signals.py

@receiver(pre_delete, sender=Post)
def delete_unused_tags(sender, instance, **kwargs):
    # Get all tags associated with the deleted post
    associated_tags = instance.tags.all()
    for tag in associated_tags:
```

```
# Only current post is linked
if tag.posts.count() == 1:
    tag.delete()
```

Define a Signal Handler for `m2m_changed`

Create a function that will be triggered whenever tags are removed from a Post (or posts are removed from a Tag). This ensures that if a Tag is no longer linked to any Post, it will be deleted.

```
# signals.py

@receiver(m2m_changed, sender=Post.tags.through)
def delete_unused_tags_on_m2m(sender, instance, action, reverse, model,
pk_set, **kwargs):
    if not reverse:
        # instance = Post, model = Tag, pk_set are Tag IDs
        if action == "pre_clear":
            instance._old_tag_ids = list(instance.tags.values_list("pk",
flat=True))

        elif action == "post_remove":
            if pk_set:
                for tag in model.objects.filter(pk__in=pk_set):
                    if tag.posts.count() == 0:
                        tag.delete()

        elif action == "post_clear":
            tag_ids = getattr(instance, "_old_tag_ids", [])
            if tag_ids:
                for tag in model.objects.filter(pk__in=tag_ids):
                    if tag.posts.count() == 0:
                        tag.delete()
            if hasattr(instance, "_old_tag_ids"):
                delattr(instance, "_old_tag_ids")

    else:
        # reverse=True: instance = Tag, model = Post, pk_set are Post IDs
        if action in ("post_remove", "post_clear"):
            if instance.posts.count() == 0:
                instance.delete()
```

Further Readings

Several Django topics are worth reading:

- [Django Source Code](#)
- [Model inheritance](#)
- [Middleware](#)
- [Template Language and Rendering](#)
- [Custom Template Tags and Filters](#)
- [QuerySet API](#)
- [Context and Context Processors](#)
- [Decorator](#)
- [Error Handling](#)
- [Django Exceptions](#)
- [Django Shell](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.

